

Dado que se requiere conocer el número de soluciones, es necesario explorar todo el árbol, incluso si se encuentra una solución rápidamente. Por lo tanto, desde el punto de vista del tiempo, no importa si se utiliza DFS o BFS. Como DFS ocupa menos espacio, es el método de búsqueda preferido para este problema.

### Viajes de ubres

La empresa de transporte de ganado Udder Travel tiene su sede en la granja A y posee un camión de transporte de ganado que utiliza para recoger y entregar vacas entre siete granjas: A, B, C, D, E, F y G. Las distancias (conmutativas) entre las granjas se representan mediante una matriz. Cada mañana, Udder Travel debe decidir, a partir de un conjunto de órdenes de traslado de ganado, el orden en que recoger y entregar las vacas para minimizar la distancia total recorrida. Estas son las reglas:

1. El camión siempre sale de la sede en la granja A y debe regresar allí cuando se hayan realizado las entregas del día.
2. El camión solo puede transportar una vaca a la vez.
3. Los pedidos se dan en pares de letras que indican dónde se debe recoger una vaca, seguido del lugar donde se debe entregar.

Tu trabajo consiste en escribir un programa que, dado un conjunto de pedidos, determine la ruta más corta que permita realizar todas las entregas, comenzando y terminando en la granja A.

Dado que es necesario probar todas las posibilidades para encontrar la mejor, se debe recorrer todo el árbol, lo cual lleva el mismo tiempo tanto con DFS como con BFS. Como DFS utiliza mucho menos espacio y es conceptualmente más fácil de implementar, es mejor usar ese método.

### Travesía del desierto

Un grupo de nómadas del desierto colabora para intentar que uno de ellos cruce el desierto. Cada nómada puede cargar una cantidad determinada de cuartos de galón de agua y bebe una cantidad determinada de agua al día, pero las cantidades que pueden cargar y las que necesitan varían. Dada la capacidad de carga y las necesidades de agua de cada nómada, determine el número mínimo de nómadas necesarios para que al menos uno pueda cruzar el desierto.

Todos los nómadas deben sobrevivir, por lo que cada nómada que emprende el viaje debe regresar en algún momento, llevando consigo suficiente agua para volver al punto de partida, o bien debe llegar al otro lado del desierto. Sin embargo, si un nómada tiene agua de sobra al momento de regresar, puede compartirla con sus compañeros, siempre y cuando estos puedan transportarla.

Este problema en realidad consta de dos problemas recursivos: uno sobre el conjunto de nómadás a utilizar y otro sobre cuándo los nómadás regresan. La búsqueda en profundidad con profundización iterativa funciona bien aquí para determinar los nómadás necesarios, probando primero si alguno puede cruzar por sí solo, luego viendo si dos colaboran para cruzar, y así sucesivamente.

### Cadenas de suma

Una cadena de sumas es una secuencia de números enteros donde el primer número es 1, y cada número subsiguiente es la suma de dos números (no necesariamente únicos) que aparecen en la lista anterior. Por ejemplo, 1 2 3 5 es una cadena de este tipo, ya que 2 es  $1+1$ , 3 es  $2+1$  y 5 es  $2+3$ . Encuentra la cadena de longitud mínima que termina con un número dado.

La búsqueda en profundidad con profundización iterativa funciona bien aquí, ya que la DFS tiende a intentar primero 1 2 3 4 5 ... n, lo cual es realmente malo y la cola crece demasiado rápido para la BFS.

### Búsqueda informada

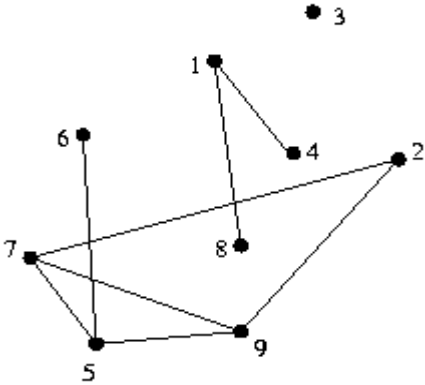
A diferencia de la búsqueda no informada, la búsqueda informada dispone de información que puede utilizarse para mejorar la selección de rutas. Ejemplos de búsqueda informada: búsqueda primero el mejor, búsqueda heurística como  $A^*$ .

### Mejor primera búsqueda

Definimos una función  $f(n) = g(n)$  donde  $g(n)$  es el valor estimado desde el nodo 'n' hasta el objetivo. Esta búsqueda es "informada" porque realizamos un cálculo para estimar  $g(n)$ .

### Búsqueda $A^*$

$f(n) = h(n) + g(n)$ , similar a la búsqueda primero en amplitud, utiliza  $g(n)$ , pero también  $h(n)$ , el costo total incurrido hasta el momento. La mejor opción de búsqueda si se sabe cómo calcular  $g(n)$ .

| Componentes                                                                       |                                                                                                                                                                                                                                                                                                                    |
|-----------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <p>Dado: un grafo no dirigido (ver imagen de la derecha)</p> <p>El <u>componente</u> de un grafo es un subgrafo de tamaño maximizado (aunque no necesariamente máximo) que está conectado.</p> <p>Calcula el componente de la gráfica.</p> <p>Este gráfico tiene tres componentes: {1,4,8}, {2,5,6,7,9} y {3}.</p> |

#### Algoritmo de llenado por inundación

**Relleno de inundación** Se puede realizar de tres maneras básicas: búsqueda en profundidad, búsqueda en amplitud y búsqueda en amplitud. La idea básica es encontrar algún nodo que no haya sido asignado a un componente y calcular el componente que contiene. La pregunta es cómo calcular el componente.

En la formulación de búsqueda en profundidad, el algoritmo examina en cada paso a todos los vecinos del nodo actual y, para aquellos que aún no han sido asignados a un componente, los asigna a dicho componente y realiza una búsqueda recursiva sobre ellos.

En la formulación de búsqueda en amplitud, en lugar de iterar sobre los nodos recién asignados, estos se agregan a una cola.

En la formulación de exploración en amplitud, cada nodo tiene dos valores: componente y visitado. Al calcular el componente, el algoritmo recorre todos los nodos que han sido asignados a ese componente pero que aún no han sido visitados, y asigna sus vecinos al componente actual.

La formulación de búsqueda en profundidad es la más fácil de programar y depurar, pero puede requerir una pila tan grande como el grafo original. Para grafos explícitos, esto no supone un gran problema, pero para grafos implícitos, como el que presenta el problema, el número de nodos puede ser muy elevado.

La formulación en amplitud funciona un poco mejor, ya que la cola es mucho más eficiente que la pila de ejecución, pero aún puede presentar el mismo problema. Tanto la formulación en profundidad como la de amplitud se ejecutan en tiempo  $N + M$ , donde  $N$  es el número de vértices y  $M$  es el número de aristas.

Sin embargo, la formulación de exploración en amplitud requiere muy poco espacio adicional. De hecho, al ser un poco compleja, no requiere espacio adicional. Sin embargo, es más lenta, requiriendo hasta  $N^2N + M$  tiempo, donde  $N$  es el número de vértices en el grafo.

### Exploración en amplitud

# component(i) denota el componente en el que se encuentra el nodo i.

1 función flood\_fill(new\_component) 2 hacer

```
3   num_visitados = 0
4   para todos los nodos i
5       si componente(i) = -2
6           num_visitados = num_visitados + 1
7           componente(i) = nuevo_componente
8           para todos los vecinos j del nodo i
9               si componente(i) = nulo
10                  componente(i) = -2
11 hasta que num_visited = 0 12
función find_components 13
    num_componentes = 0
14   para todos los nodos i
15       componente(nodo i) = nil para
16   todos los nodos i
17       si el componente (nodo i) es nulo
18           num_componentes = num_componentes + 1
19           componente(i) = -2
20   inundación_llenar(componente(num_componentes))
```

El tiempo de ejecución de este algoritmo es  $O(N^2)$ , donde  $N$  es el número de nodos. Cada arista se recorre dos veces (una vez por cada extremo) y cada nodo se marca solo una vez.

### Propiedad de la empresa

Dado: Un grafo dirigido ponderado, con pesos entre 0 y 100.

Un vértice A «posee» otro vértice B si:

1.  $A = B$
2. Hay un arco de A a B con un peso mayor a 50.
3. Existe algún conjunto de vértices  $C_1$  a  $C_k$  tales que A posee  $C_1$  a  $C_k$ , y cada vértice tiene un arco de peso  $x_1$  a  $x_k$  hasta el vértice B, y  $x_1 + x_2 + \dots + x_k > 50$ .

Encuentra todos los pares (a,b) tales que a sea dueño de b.

Esto se puede resolver adaptando el cálculo de los vértices alcanzables desde un vértice en un grafo dirigido. Para calcular qué vértices posee el vértice A, se registra el porcentaje de propiedad de cada nodo. Se inicializan todos a cero. Ahora, en cada paso recursivo, se marca el nodo como propiedad del vértice A y se suma el peso de todos los arcos salientes a los porcentajes de propiedad. Para todos los porcentajes que superen el 50%, se realiza una recursión en esos vértices.

### Carrera callejera

Datos: un grafo dirigido, un punto de inicio y un punto final.

Encuentra todos los puntos p por los que debe pasar cualquier camino desde el punto de inicio hasta el punto final.

El algoritmo más sencillo consiste en eliminar cada punto por turno y comprobar si se puede llegar al punto final desde el punto inicial. Esto tiene una complejidad temporal de  $O(N(M + N))$ . Dado que el problema original indicaba que  $M \leq 100$  y  $N \leq 50$ , este algoritmo se ejecutará fácilmente en tiempo  $O(N(M + N))$ .

## Excursiones a vacas

El diámetro de un grafo conexo se define como la distancia máxima entre dos nodos cualesquiera del grafo, donde la distancia entre dos nodos se define como la longitud del camino más corto.

Dado un conjunto de puntos en el plano y las conexiones entre esos puntos, encuentre los dos puntos que actualmente no están en el mismo componente, de manera que el diámetro del componente resultante se minimice.

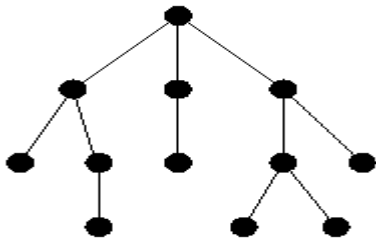
Encuentra los componentes del gráfico original, utilizando el método descrito anteriormente. Luego, para cada par de puntos que no pertenezcan al mismo componente, intenta conectarlos. Encuentra el par que minimice el diámetro.

## Campos conectados

El granjero John contrató la construcción de un nuevo granero. Desafortunadamente, el constructor confundió los planos del granero de John con otros. Los planos de John contemplaban un granero de una sola habitación, pero el edificio que recibió podría tener varias. Con la ayuda de una cuadrícula que muestre la distribución del granero, dile al granjero John cuántas habitaciones tiene.

Análisis: El gráfico aquí se encuentra en las ubicaciones de la cuadrícula que no son paredes, con una arista entre ubicaciones adyacentes que no son paredes, aunque el gráfico debería almacenarse como la cuadrícula y no transformarse en alguna otra forma, ya que la cuadrícula es muy compacta y fácil de usar.

## Árbol



Un árbol es una de las estructuras de datos más eficientes utilizadas en un programa informático. Existen muchos tipos de árboles. Un árbol binario es un árbol que siempre tiene dos ramas, árboles rojo-negro, árboles 2-3-4, árboles AVL, etc. Un árbol bien equilibrado puede utilizarse para diseñar un buen algoritmo de búsqueda. Un árbol es un grafo no dirigido que no contiene ciclos y está conectado. Muchos árboles son lo que se denomina enraizado donde existe la noción de "la cima"

nodo, que se llama raíz. Por lo tanto, cada nodo tiene uno padre, que es el nodo adyacente que está más cerca de la raíz, y puede tener cualquier número de niños, que son el resto de los nodos adyacentes a él. El árbol de arriba se dibujó como un árbol con raíz.

## Árboles de expansión mínima

### Árboles que se extienden

Un árbol de expansión de un grafo es simplemente un subgrafo que contiene todos los vértices y es un árbol. Un grafo puede tener muchos árboles de expansión; por ejemplo, el grafo completo de cuatro vértices.

```

O---O
|\ /|
| X |
|/ \|
O---O

```

tiene dieciséis árboles de expansión:

```

O---O  O---O  OO  O---O
| | | | | | |
| | | | | | |
O  O  O---O  O---O  O---O

```

```

O---O  O  O  OO  OO
\ /  | \ /  \ /  \ / |
incógnita  X  incógnita  X |
/ \  | / \  / \  / \ |
OO  O  O  O---O  OO

```

```

O  O  O---O  O  O  O---O
| \ |  /  | / |  \
| \ |  /  | / |  \
| \ |  /  | / |  \
O  O  O---O  O  O  O---O

```

```

O---O  O  OOO  O---O
| \ |  /  \ | / |
| \ |  /  \ | / |
| \ |  /  \ | / |
O  O  O---O  O---OO  O

```

## árboles de expansión mínima

Supongamos ahora que las aristas del grafo tienen pesos o longitudes. El peso de un árbol es simplemente la suma de los pesos de sus aristas. Obviamente, distintos árboles tienen longitudes distintas. El problema es: ¿cómo encontrar el árbol de expansión de longitud mínima?

### ¿Por qué utilizar árboles de expansión mínima?

La aplicación estándar se da a problemas como el diseño de redes telefónicas. Imagina una empresa con varias oficinas; quieres alquilar líneas telefónicas para conectarlas entre sí; y la compañía telefónica cobra precios diferentes para conectar distintas ciudades. Buscas un conjunto de líneas que conecte todas tus oficinas con el menor coste posible. Debería ser un árbol de expansión, ya que si una red no lo es, siempre puedes eliminar algunas aristas y ahorrar dinero.

Una aplicación menos obvia es que el árbol de expansión mínima puede utilizarse para resolver aproximadamente el problema del viajante. Una forma formal y práctica de definir este problema consiste en encontrar el camino más corto que visite cada punto al menos una vez.

Tenga en cuenta que si un camino visita todos los puntos exactamente una vez, se trata de un tipo especial de árbol. Por ejemplo, en el ejemplo anterior, doce de los dieciséis árboles de expansión son en realidad caminos. Si un camino visita algunos vértices más de una vez, siempre puede eliminar algunas aristas para obtener un árbol. Por lo tanto, en general, el peso del MST es menor que el del TSP, ya que se trata de una minimización sobre un conjunto estrictamente mayor.

Por otro lado, si trazas un camino alrededor del árbol de expansión mínima, recorres cada arista dos veces y visitas todos los puntos, por lo que el peso del TSP es menor que el doble del peso del MST. Por lo tanto, este recorrido se encuentra a un factor de dos del óptimo.

### ¿Cómo encontrar el árbol de expansión mínima?

El método absurdo consiste en listar todos los árboles de expansión y encontrar el mínimo de la lista. Ya sabemos cómo encontrar mínimos... Pero hay demasiados árboles para que esto sea eficiente. Además, no es realmente un algoritmo, porque aún se necesitaría saber cómo listar todos los árboles.

Una mejor idea es encontrar alguna propiedad clave del MST que nos permita estar seguros de que alguna arista forma parte de él, y usar esta propiedad para construir el MST arista por arista.



Para simplificar, asumimos que existe un único árbol de expansión mínima. Si bien es posible que ideas como esta funcionen sin esta suposición, resulta más difícil enunciar los teoremas o escribir los algoritmos con precisión.

**Lema** Sea  $X$  un subconjunto cualquiera de los vértices de  $G$ , y sea la arista  $e$  la arista más corta que conecta  $X$  con  $G \setminus X$ . Entonces  $e$  forma parte del árbol de expansión mínima.

**Prueba** Supongamos que tenemos un árbol  $T$  que no contiene a  $e$ ; entonces queremos demostrar que  $T$  no es el MST. Sea  $e=(u,v)$ , con  $u$  en  $X$  y  $v$  fuera de  $X$ . Entonces, como  $T$  es un árbol de expansión, contiene un camino único de  $u$  a  $v$ , que junto con  $e$  forma un ciclo en  $G$ . Este camino debe incluir otra arista  $f$  que conecta  $X$  con  $G \setminus X$ .  $T+ef$  es otro árbol de expansión (tiene el mismo número de aristas y permanece conectado, ya que se puede reemplazar cualquier camino que contenga  $f$  por uno que recorra el ciclo en sentido contrario). Tiene menor peso que  $t$ , ya que  $e$  tiene menor peso que  $f$ . Por lo tanto,  $T$  no era el mínimo, que es lo que queríamos demostrar.

### El algoritmo de Kruskal

Comenzaremos con el algoritmo de Kruskal, que es el más fácil de entender y probablemente el mejor para resolver problemas manualmente.

Algoritmo de Kruskal:  
ordenar las aristas de  $G$  en orden creciente por longitud  
mantener un subgrafo  $S$  de  $G$ , inicialmente vacío  
para cada arista  $e$  en orden ordenado  
    Si los extremos de  $e$  están desconectados en  $S$ , agregue  $e$   
    a  $S$ .  
devolver  $S$

Tenga en cuenta que, siempre que agregue una arista  $(u,v)$ , siempre es la más pequeña que conecta la parte de  $S$  alcanzable desde  $u$  con el resto de  $G$ , por lo que según el lema debe ser parte del MST.

Este algoritmo se conoce como un *algoritmo codicioso* porque elige en cada paso la arista más barata para añadir a  $S$ . Debes tener mucho cuidado al intentar usar algoritmos voraces para resolver otros problemas, ya que normalmente no funcionan. Por ejemplo, si quieres encontrar el camino más corto de  $a$  a  $b$ , podría ser una mala idea seguir tomando las aristas más cortas. La idea voraz solo funciona en el algoritmo de Kruskal debido a la propiedad clave que demostramos.

**Análisis** La línea que comprueba si dos extremos están desconectados parece que debería ser lenta (tiempo lineal por iteración, o  $O(mn)$  en total). La parte más lenta resulta ser el paso de ordenación, que tarda  $O(m \log n)$  tiempo.

### El algoritmo de Prim

En lugar de construir un subgrafo arista por arista, el algoritmo de Prim construye un árbol vértice por vértice.

```

El algoritmo de Prim:
Sea T un único vértice x
mientras (T tiene menos de n vértices) {
    encuentra la arista más pequeña que conecta T con GT y
    agrégala a T
}

```

Ejemplo del algoritmo de Prim en lenguaje C:

```

/* usedp=>cuántos puntos ya se han usado
p->array de estructuras, que consiste en x, y, & usadas/no usadas. Este
problema consiste en obtener el MST de un grafo con n vértices donde el peso
de una arista es la distancia entre 2 puntos */

usedp=p[0].used=1; /* seleccionar un punto arbitrario como punto de partida */ while
(usedp<n) {
    pequeño=-1.0;

    para (i=0;i<n;i++) si (p[i].usado)
        para (j=0;j<n;j++) si (!p[j].usado) {
            longitud=sqrt(pow(p[i].xp[j].x,2) + pow(p[i].yp[j].y,2));

            si (pequeño== -1.0 || longitud<pequeño) {
                pequeño=longitud;
                p pequeño = j;
            }
        }
    minLength+=pequeño;

    p[smallp].usado=1;
    usedp++ ;
}

```

### Encontrar caminos más cortos usando BFS

Esto solo se aplica a grafos con aristas no ponderadas; simplemente realice una búsqueda en amplitud (BFS) desde el nodo inicial hasta el nodo final, y detenga la búsqueda cuando encuentre la primera aparición del nodo final.

**Elrelajación**El proceso actualiza los costos de todos los vértices,**v**, conectado a un vértice,**tú**, si pudiéramos mejorar la mejor estimación del camino más corto **aval** incluir(**u,v**)en el camino hacia**v**El procedimiento de relajación se desarrolla de la siguiente manera:

```

inicializar_fuente_única(Grafo G, Nodo s)
  para cada vértice v en Vértices(G)
    Gd[v]:=infinito
    G.pi[v]:=nil
  Gd[s]:=0;

```

Esto configura el gráfico de manera que cada nodo no tenga predecesor (**pi[v] = nil**) y las estimaciones del costo (distancia) de cada nodo desde la fuente (**d[v]**) son infinitos, excepto el nodo fuente mismo (**d[s] = 0**).

Tenga en cuenta que también hemos introducido otra forma de almacenar un grafo (o parte de un grafo, ya que esta estructura solo puede almacenar un árbol de expansión), el **subgrafo predecesor**-la lista de predecesores de cada nodo, **pi[j]**,  $1 \leq j \leq |V|$ . Las aristas en el subgrafo predecesor son (**pi[v], v**).

El procedimiento de relajación comprueba si la mejor estimación actual de la distancia más corta **av(d[v])** se puede mejorar pasando por **tú** (*es decir* haciendo **tú** el predecesor de **v**):

```

relax(Nodo u, Nodo v, double w[u,v])
  si d[v] > d[u] + w[u,v] entonces
    d[v]:=d[u] + w[u,v]
    pi[v]:=u

```

### Algoritmo de Dijkstra

El algoritmo de Dijkstra (inventado por Edsger W. Dijkstra) resuelve el problema de encontrar el camino más corto desde un punto en un grafo (el *fuentes*) a un destino. Resulta que se pueden encontrar los caminos más cortos desde un origen dado hasta *todos* los puntos en un gráfico en el mismo tiempo, por lo tanto este problema se llama **Rutas más cortas desde un único origen** problema.

Tampoco habrá ciclos, ya que un ciclo definiría más de un camino desde el vértice seleccionado hasta al menos otro vértice. Para un grafo, **G=(V,E)** donde **V** es un conjunto de vértices y **E** es un conjunto de aristas.

El algoritmo de Dijkstra mantiene dos conjuntos de vértices: **S** (el conjunto de vértices cuyos caminos más cortos desde el origen ya han sido determinados) y **V-S** (los vértices restantes). Las demás estructuras de datos necesarias son: **d** (conjunto de mejores estimaciones del camino más corto a cada vértice) y **pi** (una serie de predecesores para cada vértice)

El modo de funcionamiento básico es:

1. Inicializar d y pi,
2. Establezca S como vacío,

3. Mientras aún existan vértices en VS,
4. Ordena los vértices en VS según la mejor estimación actual de su distancia desde la fuente,
5. Añade u, el vértice más cercano en VS, a S,
6. Relaja todos los vértices que aún están en VS conectados a u

```

DIJKSTRA(Grafo G,Nodo s)
  inicializar_fuente_única(G,s) S:={ 0 }
  /* Vaciar S */
  Q:=Vértices(G) /* Coloca los vértices en una cola de prioridad */
  mientras no esté vacía(Q)
    u:=ExtraerMínimo(Q);
    AddNode(S,u); /* Agregar u a S */
    para cada vértice v que es adyacente a u
      relajar(u,v,w)

```

### Algoritmo de Bellman-Ford

Un algoritmo más generalizado para encontrar el camino más corto desde un único origen, capaz de hallar el camino más corto en un grafo con aristas de peso negativo. Si no existe ningún ciclo negativo en el grafo, este algoritmo actualiza cada  $d[v]$  con el camino más corto desde  $s$  hasta  $v$ , completa la lista de predecesores "pi" y devuelve VERDADERO. Sin embargo, si existe un ciclo negativo en el grafo, este algoritmo devuelve FALSO.

```

BELLMAN_FORD(Grafo G,double w[],[],Nodo s)
  inicializar_fuente_única(G,s) para i=1
  hasta |V[G]|-1
    para cada arista (u,v) en E[G]
      relajar(u,v,w)

  para cada arista (u,v) en E[G]
    Si  $d[v] > d[u] + w(u, v)$  entonces
      devolver FALSO

  devolver VERDADERO

```

#### PON A PRUEBA TUS CONOCIMIENTOS SOBRE BELLMAN FORD

Resuelva los problemas del juez en línea de Valladolid relacionados con Bellman Ford:

558 - [Agujeros de gusano](#), simplemente compruebe la existencia del ciclo negativo.

### Rutas más cortas desde un único origen en un grafo acíclico dirigido (DAG)

Existe un algoritmo más eficiente para resolver el problema del camino más corto desde un único origen en un grafo acíclico dirigido (DAG). Por lo tanto, si sabe con certeza que su grafo es un DAG, quizás le convenga considerar este algoritmo en lugar de usar el de Dijkstra.

```
DAG_SHORTEST_PATHS(Grafo G,double w[],Nodo s)
ordenar_topológicamente_los_vértices_de_G // O(V+E)
inicializar_fuente_única(G,s)

para_cada_vértice_u_tomado_en_orden_topológicamente_ordenado
para_cada_vértice_v_que_es_adyacente_a_u
relajar(u,v,w)
```

Un ejemplo de aplicación de este algoritmo DAG\_SHORTEST\_PATHS (tal como se presenta en el libro CLR) es resolver el problema de la ruta crítica, es decir, encontrar la ruta más larga a través de un DAG, por ejemplo: calcular el tiempo más rápido para completar una tarea compleja que consta de tareas más pequeñas cuando se conoce el tiempo necesario para completar cada tarea pequeña y el orden de precedencia de las tareas.

### Floyd Warshall

Dado un grafo dirigido, el algoritmo de Floyd-Warshall para encontrar los caminos más cortos entre todos los pares de nodos calcula los caminos más cortos en  $O(n^3)$ . En esta página, presentamos el algoritmo de Floyd-Warshall y su variante, junto con sus códigos fuente.

**Dado:**

**w:** pesos de los bordes **d :**

matriz de distancias **pag :**

matriz predecesora

$w[i][j]$  = longitud de la arista directa entre  $i$  y  $j$   $d[i]$

$[j]$  = longitud del camino más corto entre  $i$  y  $j$

$p[i][j]$  = en el camino más corto de  $i$  a  $j$ ,  $p[i][j]$  es el último nodo antes de  $j$ .

#### Inicialización

```
para (i=0; i<n; i++)
para (j=0; j<n; j++) {
d[i][j] = w[i][j];
p[i][j] = i;
}
para (i=0; i<n; i++) d[i][i] = 0;
```

## El algoritmo

```

para (k=0;k<n;k++) /* k -> es el punto intermedio */
  para (i=0;i<n;i++) /* empezar desde i */
    para (j=0;j<n;j++) /* alcanzando j */
      /* si i->k + k->j es menor que el i->j original */ si (d[i][k] + d[k][j] < d[i][j]) {

          /* luego reduce la distancia i->j a la menor i->k->j */ graph[i][j] = graph[i][k]
          +graph[k][j];
          /* y actualiza la matriz predecesora */ p[i][j] = p[k][j];

      }

```

En la  $k$ -ésima iteración del bucle externo, intentamos mejorar los caminos más cortos conocidos hasta el momento considerando  $k$  como un nodo intermedio. Por lo tanto, después de la  $k$ -ésima iteración conocemos aquellos caminos más cortos que solo contienen nodos intermedios del conjunto  $\{0, 1, 2, \dots, k\}$ . Después de las  $n$  iteraciones, conocemos los caminos más cortos reales.

### Construcción de un camino más corto

```

print_path (int i, int j) {
    si (i!=j)      print_path(i,p[i][j]);
    imprimir(j);
}

```

#### PON A PRUEBA TUS CONOCIMIENTOS SOBRE FLOYD WARSHALL

Resuelva los problemas relacionados con el juez en línea de Valladolid y Floyd Warshall:

[104 - Arbitraje](#) - Modificar correctamente el parámetro de Floyd-Warshall [423](#)  
[- MPI Maelstrom](#)  
[436 - Arbitraje \(II\)](#) - Modificar correctamente el parámetro Floyd Warshall  
[567 - Riesgo](#) - aunque se puede resolver mediante fuerza bruta

## Casco transitivo

Dado un grafo dirigido, el algoritmo de Floyd-Warshall puede calcular la envolvente transitiva en  $O(n^3)$ . Transitiva significa que si  $i$  puede alcanzar  $k$  y  $k$  puede alcanzar  $j$ , entonces  $i$  puede alcanzar  $j$ . En la envolvente transitiva, se calcula la alcanzabilidad de todos los vértices.

**w**:matriz de adyacencia

**d**:casco transitivo

$w[i][j]$  = arista entre  $i$  y  $j$  (0=sin arista, 1=arista)  $d[i][j]$  = 1 si y solo si  $j$  es alcanzable desde  $i$

### Inicialización

```
para (i=0; i<n; i++)
  para (j=0; j<n; j++)
    d[i][j] = w[i][j];

para (i=0; i<n; i++)
  d[i][i] = 1;
```

### El algoritmo

```
para (k=0; k<n; k++)
  para (i=0; i<n; i++)
    para (j=0; j<n; j++)
      /* d[i][j] es verdadero si d[i][j] ya es verdadero
        o si podemos usar k como vértice intermedio para llegar a j desde i, de lo
        contrario, d[i][j] es falso */
      d[i][j] = d[i][j] || (d[i][k] && d[k][j]);
```

#### PON A PRUEBA TUS CONOCIMIENTOS SOBRE EL HULL FLOYD WARSHALL (VERSIÓN TRANSITIVA)

Resuelva problemas de Valladolid Online Judge relacionados con la envoltura transitiva: [334 - Identificación de eventos concurrentes](#) - La parte interna de este problema necesita una envoltura transitiva, aunque este problema es más complejo que eso.

## Distancia MiniMax

Dado un grafo dirigido con longitudes de arista, el algoritmo de Floyd-Warshall puede calcular la distancia minimax entre cada par de nodos en  $O(n^3)$ . Para ver un ejemplo de un problema minimax, consulte el problema OJ de Valladolid a continuación.

**w**: pesos de los bordes

**d**: matriz de distancia minimax

**pag**: matriz predecesora

$w[i][j]$  = longitud de la arista directa entre  $i$  y  $j$   $d[i][j]$

$[j]$  = longitud del camino minimax entre  $i$  y  $j$

### Inicialización

```
para (i=0; i<n; i++)
    para (j=0; j<n; j++)
        d[i][j] = w[i][j];

para (i=0; i<n; i++)
    d[i][i] = 0;
```

### El algoritmo

```
para (k=0; k<n; k++)
    para (i=0; i<n; i++)
        para (j=0; j<n; j++)
            d[i][j] = min(d[i][j], max(d[i][k], d[k][j]));
```

#### PON A PRUEBA TUS CONOCIMIENTOS SOBRE MINIMAX FLOYD WARSHALL

Resuelve problemas de Valladolid Online Judge relacionados con MiniMax:

[534 - Frogger](#) - seleccionar el mínimo de saltos más largos

[10048 - Audiofobia](#) - seleccione el mínimo o máximo decibelios a lo largo del recorrido

#### Distancia máxima/mínima

También puedes calcular la distancia maximin con el algoritmo de Floyd-Warshall. Maximin es el inverso de minimax. Consulta el problema de Valladolid OJ que se presenta a continuación para comprender maximin.

**w**: pesos de los bordes

**d**: matriz de distancia maximin

**pag**: matriz predecesora

$w[i][j]$  = longitud de la arista directa entre  $i$  y  $j$   $d[i]$

$[j]$  = longitud del camino maximin entre  $i$  y  $j$



**Inicialización**

```

para (i=0; i<n; i++)
    para (j=0; j<n; j++)
        d[i][j] = w[i][j];

para (i=0; i<n; i++)
    d[i][i] = 0;

```

**El algoritmo**

```

para (k=0; k<n; k++)
    para (i=0; i<n; i++)
        para (j=0; j<n; j++)
            d[i][j] = max(d[i][j], min(d[i][k], d[k][j]));

```

**PON A PRUEBA TUS CONOCIMIENTOS SOBRE MAXIMIN FLOYD WARSHALL**

Resuelve problemas de Valladolid Online Judge relacionados con MaxiMin:

[544 - Carga pesada](#) - seleccione el peso máximo o mínimo permitido a lo largo del recorrido. [10099 - La guía turística](#) - Seleccione el número máximo y mínimo de pasajeros a lo largo de la ruta y luego divida el número total de pasajeros entre este valor para determinar cuántos viajes son necesarios.

**Camino más seguro**

Dado un grafo dirigido donde las aristas están etiquetadas con probabilidades de supervivencia, se puede calcular el camino más seguro entre dos nodos (es decir, el camino que maximiza el producto de las probabilidades a lo largo del camino) con el algoritmo de Floyd-Warshall.

**w:** pesos de los bordes **pag :**

matriz de probabilidad

$w[i][j]$  = probabilidad de supervivencia de la arista entre  $i$  y  $j$   
 $p[i][j]$  = probabilidad de supervivencia del camino más seguro entre  $i$  y  $j$

**Inicialización**

```

para (i=0; i<n; i++)
    para (j=0; j<n; j++)
        p[i][j] = w[i][j];

para (i=0; i<n; i++)
    p[i][i] = 1;

```

## El algoritmo

```
para (k=0; k<n; k++)
  para (i=0; i<n; i++)
    para (j=0; j<n; j++)
      p[i][j] = max(p[i][j], p[i][k] * p[k][j]);
```

### Transposición de grafos

**Aporte** : grafo dirigido  $G = (V, E)$

**Producción** : gráfico  $G_T = (V, E_T)$ , donde  $E_T = \{(v, u) \text{ en } V \times V : (u, v) \text{ en } E\}$ .

es decir  $G_T$  es  $G$  con todas sus aristas invertidas.

Describa algoritmos eficientes para calcular  $G_T$  a partir de  $G$ , tanto para la representación de lista de adyacencia como para la de matriz de adyacencia de  $G$ . Analice los tiempos de ejecución de sus algoritmos.

Utilizando la representación de lista de adyacencia, el array  $B$  es el nuevo array de la lista de adyacencia  $G_T$

```
para (i=1; i<=p; i++)
  B[i] = nulo;

para (i=1; i<=p; i++)
  repetir {
    agregar i al final de la lista enlazada B[A[i]]; obtener el
    siguiente A[i];
  } hasta que A[i] = nil;
```

### Ciclo euleriano y trayectoria euleriana

#### Ciclo de Euler

**Aporte** : Grafo dirigido y conectado  $G = (V, E)$

**Producción** : Un ciclo que recorre cada arista de  $G$  exactamente una vez, aunque puede visitar un vértice más de una vez.

Teorema: Un grafo dirigido posee un ciclo euleriano si y solo si

- 1) Es conectado
- 2) Para todo  $\{v\}$  en  $\{V\}$ , grado de entrada( $v$ ) = grado de salida( $v$ )

#### Camino de Euler

**Aporte** : Grafo dirigido y conectado  $G = (V, E)$

**Producción** : Un camino desde  $v_1$  hasta  $v_2$ , que recorre cada arista de  $G$  exactamente una vez, aunque puede visitar un vértice más de una vez.

Teorema: Un grafo dirigido posee un camino euleriano si y solo si

1) Está conectado

2) Para todo  $\{v\}$  en  $\{V\}$   $\text{indegree}(v) = \text{outdegree}(v)$  con la posible excepción de dos vértices  $v_1, v_2$  en cuyo caso,

a) grado de entrada( $v_1$ ) = grado de salida( $v_2$ ) + 1

b) grado de entrada( $v_2$ ) = grado de salida( $v_1$ ) - 1

### Ordenación topológica

**Aporte** : Un grafo acíclico dirigido (DAG)  $G = (V, E)$

**Producción** : Un ordenamiento lineal de todos los vértices en  $V$  tal que si  $G$  contiene una arista  $(u, v)$ , entonces  $u$  aparece antes que  $v$  en el ordenamiento.

Si se representa en un diagrama, la ordenación topológica puede verse como vértices a lo largo de una línea horizontal, donde todas las aristas dirigidas van de izquierda a derecha. Un grafo dirigido  $G$  es acíclico si y solo si una búsqueda en profundidad (DFS) de  $G$  no produce ninguna arista de retroceso.

#### Ordenación topológica(G)

1. Llamar a DFS( $G$ ) para calcular los tiempos de finalización  $f[v]$  para cada vértice  $v$ .
2. A medida que se termina cada vértice, insértelo al principio de una lista enlazada.
3. Devuelve la lista enlazada de vértices.

El algoritmo de ordenación topológica se ejecuta en  $O(V+E)$  debido a la búsqueda en profundidad (DFS).

### Componentes fuertemente conectados

**Aporte** : Un grafo dirigido  $G = (V, E)$

**Producción** : Todos los componentes fuertemente conectados de  $G$ , donde en un componente fuertemente conectado, todos los pares de vértices  $u$  y  $v$  en ese componente, tenemos  $u \rightsquigarrow v$  y  $v \rightsquigarrow u$ , es decir,  $u$  y  $v$  son alcanzables entre sí.

#### Componentes fuertemente conectados (G)

1. Llamar a DFS( $G$ ) para calcular los tiempos de finalización  $f[u]$  para cada vértice  $u$ .
2. Calcular  $GT$ , invirtiendo todas las aristas en  $G$  usando la lista de adyacencia.
3. Llamar a DFS( $GT$ ), pero en el bucle principal de DFS, considerar los vértices en orden decreciente de  $f[u]$  como se calculó en el paso 1.
4. Generar los vértices de cada árbol en el bosque de búsqueda en profundidad del paso 3 como un componente fuertemente conectado separado.

El algoritmo Strongly-Connected-Components se ejecuta en  $O(V+E)$ .

## CAPÍTULO 13 GEOMETRÍA COMPUTACIONAL

La geometría computacional es una materia importante. Dominar esta materia puede ayudarte en concursos de programación, ya que cada concurso suele incluir 1 o 2 problemas geométricos. problemas.[2]

### Objetos geométricos y sus propiedades

#### sistema de coordenadas terrestres

La gente usa *latitudes* (líneas horizontales) y *longitudes* (líneas verticales) en el sistema de coordenadas terrestres.

La longitud abarca desde 0 grados (Greenwich) hasta +180° Este y -180° Oeste. La latitud abarca desde 0 grados (Ecuador) hasta +90° (Polo Norte) y -90° (Polo Sur).

La pregunta más interesante es cuál es la distancia esférica/geográfica entre dos ciudades p y q en la Tierra con radio r, denotada por (p\_lat, p\_long) a (q\_lat, q\_long). Todas las coordenadas están en radianes. (Es decir, convertir el rango de longitud [-180..180] y el rango de latitud [-90..90] a [-pi..pi] respectivamente).

Tras derivar las ecuaciones matemáticas, la respuesta es la siguiente:

$$\text{distancia\_esférica}(p\_lat, p\_long, q\_lat, q\_long) = \text{acos}(\sin(p\_lat) * \sin(q\_lat) + \cos(p\_lat) * \cos(q\_lat) * \cos(p\_long - q\_long)) * r$$

Dado que  $\cos(ab) = \cos(a)\cos(b) + \sin(a)\sin(b)$ , podemos simplificar la fórmula anterior a:

$$\begin{aligned} \text{distancia\_esférica}(p\_lat, p\_long, q\_lat, q\_long) = & \text{acos}(\sin(p\_lat) * \\ & \sin(q\_lat) + \\ & \cos(p\_lat) * \cos(q\_lat) * \cos(p\_long) * \cos(q\_long) + \cos(p\_lat) * \cos(q\_lat) \\ & * \sin(p\_long) * \sin(q\_long)) * r \end{aligned}$$

#### PON A PRUEBA TUS CONOCIMIENTOS DEL SISTEMA DE COORDENADAS TERRESTRES

Resuelve problemas de UVA relacionados con el Sistema de Coordenadas Terrestres:

[535 - Trotamundos](#)

[10075 - Aerolíneas](#) - combinadas con la ruta más corta entre todos los pares

### Envolvente convexa

Básicamente, el problema de la envoltura convexa es el más básico y popular de la geometría computacional. Existen numerosos algoritmos para resolverlo de forma eficiente, con una cota inferior óptima de  $O(n \log n)$ . Esta cota inferior ya ha sido demostrada.

**Problema de la envoltura convexa (versión 2D):**

**Aporte:** Un conjunto de puntos en el plano euclidiano

**Producción:** Encuentra el conjunto mínimo de puntos que encierren a todos los demás puntos.

**Algoritmos de envolvente convexa:**

- a. Jarvis March / Envoltura de regalos
- b. Escaneo de Graham
- c. Casco rápido
- d. Divide y vencerás

CAPÍTULO 14 CATEGORÍA DEL PROBLEMA VALLADOLID Oj<sub>[2]</sub>

| Matemáticas                                          |                                                                                                                                                                                                                             |
|------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>(General)</b>                                     | 113,202,256,275,276,294326,332,347, 350,356,374,377,382,386, 412,465,471,474,485,498550,557,568,594,725,727,846,10006,10014,10019,10042,10060,10071,10093,10104,10106,10107,10110,10125,10127,10162,10190,10193,10195,10469 |
| <b>Números primos</b>                                | 406,516,543,583,686,10140,10200,10490                                                                                                                                                                                       |
| <b>Geometría</b>                                     | 190, 191, 378, 438, 476, 477, 478, 10112, 10221, 10242, 10245, 10301, 10432, 10451                                                                                                                                          |
| <b>Grandes números</b>                               | 324,424,495,623,713,748,10013,10035,10106,10220, 10334                                                                                                                                                                      |
| <b>Números base</b>                                  | 343,355,389,446,575,10183,10551                                                                                                                                                                                             |
| <b>Combinaciones / Permutaciones</b>                 | 369.530                                                                                                                                                                                                                     |
| <b>Teorías / Fórmulas</b>                            | 106.264.486.580                                                                                                                                                                                                             |
| <b>Factorial</b>                                     | 160, 324, 10323, 10338                                                                                                                                                                                                      |
| <b>Fibonacci</b>                                     | 495,10183,10334,10450                                                                                                                                                                                                       |
| <b>Secuencias</b>                                    | 138.10408                                                                                                                                                                                                                   |
| <b>Módulo</b>                                        | 10176,10551                                                                                                                                                                                                                 |
| Programación dinámica                                |                                                                                                                                                                                                                             |
| <b>General</b>                                       | 108, 116, 136, 348, 495, 507, 585, 640, 836, 10003, 10036, 10074, 10130, 10404                                                                                                                                              |
| <b>El más largo Aumento/Disminución Subsecuencia</b> | 111.231.497.10051.10131                                                                                                                                                                                                     |
| <b>Común más largo Subsecuencia</b>                  | 531, 10066, 10100, 10192, 10405                                                                                                                                                                                             |
| <b>Contando el cambio</b>                            | 147, 357, 674                                                                                                                                                                                                               |
| <b>Editar distancia</b>                              | 164.526                                                                                                                                                                                                                     |
| Gráfico                                              |                                                                                                                                                                                                                             |
| <b>Floyd Warshall Todo-Pares más cortos Camino</b>   | 1043, ,436,534,544,567,10048,10099,10171,112,117,122,193, 336,352,383, 429,469, 532, 536, 590, 614, 615, 657, 677, 679, 762,785,10000,10004,10009,10010,10116,10543                                                         |
| <b>Flujo de red</b>                                  | 820, 10092, 10249                                                                                                                                                                                                           |
| <b>Máximo bipartito Pareo</b>                        | 670.753.10080                                                                                                                                                                                                               |
| <b>Relleno de inundación</b>                         | 352.572                                                                                                                                                                                                                     |
| <b>Punto de articulación</b>                         | 315.796                                                                                                                                                                                                                     |
| <b>MST</b>                                           | 10034,10147,10397                                                                                                                                                                                                           |
| <b>Hallazgo de la Unión</b>                          | 459.793.10507                                                                                                                                                                                                               |
| <b>Ajedrez</b>                                       | 167.278.439.750                                                                                                                                                                                                             |
| Problemas mixtos                                     |                                                                                                                                                                                                                             |
| <b>Anagrama</b>                                      | 153.156.195.454.630                                                                                                                                                                                                         |
| <b>Clasificación</b>                                 | 120,10152,10194,10258                                                                                                                                                                                                       |
| <b>Cifrado</b>                                       | 458,554,740,10008,10062                                                                                                                                                                                                     |
| <b>Algoritmo voraz</b>                               | 10020,10249,10340                                                                                                                                                                                                           |
| <b>Juego de cartas</b>                               | 162.462.555                                                                                                                                                                                                                 |
| <b>Analizador BNF</b>                                | 464.533                                                                                                                                                                                                                     |
| <b>Simulación</b>                                    | 130,133,144, 151, 305, 327,339,362,379,402,440,556,637,758,10033,10500                                                                                                                                                      |
| <b>Relacionado con la producción</b>                 | 312,320, 330, 337, 381,391,392,400,403,445,488,706,100,82                                                                                                                                                                   |

|                                         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|-----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>A propósito</b>                      | 101,102,103, 105,118, 119,121,128, 142,145,146,154, 155,187,195220,227,232, 271,272,291,297,299,300,311,325,333,335,340,344,349,353,380,384,394,401,408,409,413,414,417,434,441,442,444,447,455,457,460,468,482,483,484,489,492,494,496,499537,541,542,551,562,573,574, 576,579,586,587,591602,612,616,617,620,621,642,654,656,661,668,671,673729,755,837,10015,10017,10018,10019,10025,10038, 10041,10045,10050,10055,10070,10079,10098,10102, 10126, 10161,10182,10189, 10281,10293, 10487 |
| <b>Formación</b><br><b>Manipulación</b> | 466,10324,10360,10443                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Búsqueda binaria</b>                 | 10282,10295,10474                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <b>Retroceder</b>                       | 216,291422,524,529, 539, 571, 572, 574,10067,10276,10285,10301,10344,10400,10422,10452                                                                                                                                                                                                                                                                                                                                                                                                       |
| <b>Problema 3n+1</b>                    | 100.371.694                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |

Estos problemas están disponibles en (<http://acm.uva.es/p>). Se añaden nuevos problemas después de cada concurso en línea en Valladolid Online Judge, así como después de cada concurso regional de programación de ACM. añadido al archivo en vivo de ACM (<http://cii-judge.baylor.edu/>).

## APÉNDICE A

PROBLEMAS DE PROGRAMACIÓN DE ACM

---



Esta sección del libro contiene algunos problemas interesantes de ACM/ICPC. Los problemas se han recopilado del Valladolid Online Judge. Para encontrar más fuentes de problemas, consulte la sección de referencias.



**¡Encuentra las maneras!****El problema**

Un estadounidense, un francés y una inglesa habían estado en Daca, la capital de Bangladés. Hicieron turismo en taxi. Los tres turistas conversaban sobre los lugares de interés de la ciudad. El estadounidense estaba muy orgulloso de los rascacielos de Nueva York. Presumió ante sus amigos: "¿Sabían que el Empire State Building se construyó en tres meses?".

—¿De verdad? —respondió el francés—. ¡La Torre Eiffel de París se construyó en tan solo un mes! (Sin embargo, la verdad es que la construcción de la torre comenzó en enero de 1887. Cuarenta ingenieros y diseñadores, bajo la dirección de Eiffel, trabajaron durante dos años. La torre se terminó en marzo de 1889).

—¡Qué interesante! —dijo la inglesa—. ¡El Palacio de Buckingham en Londres se construyó en tan solo dos semanas!

En ese momento, el taxi pasó junto a un gran barrio marginal (aunque en Bangladesh lo llamamos "Bostii"). "¿Qué era eso? ¿Cuándo se construyó?", preguntó la inglesa al conductor, que era bangladésí.

—¡No lo sé! —respondió el conductor—. ¡No estaba allí ayer!

Sin embargo, en Bangladesh, la proliferación de barrios marginales ilegales es un grave problema. El gobierno intenta dismantelar estos asentamientos y reubicar a sus habitantes en lugares lejanos, concretamente en aldeas planificadas a las afueras de la ciudad. Pero no encuentran la manera de eliminarlos por completo.

Ahora, ¿puedes imaginarte a ti mismo como un destructor de barrios marginales? ¿De cuántas maneras puedes destruir ~~los~~ barrios marginales fuera de ~~la~~ ~~norte~~ Barrios marginales! Supongamos que hay 10 barrios marginales y te dan permiso para destruir 5 de ellos. Seguramente puedes hacerlo de 252 maneras, que es un número de solo 3 dígitos. ¡Tu tarea es encontrar los dígitos de las maneras en que puedes destruir los barrios marginales!

**La entrada**

El archivo de entrada contendrá uno o más casos de prueba. Cada caso de prueba consta de una línea que contiene dos números enteros  $n$  ( $n \geq 1$ ) y  $k$  ( $1 \leq k \leq n$ ).

**Entrada de ejemplo**

20 5  
100 10  
200 15

**Salida de ejemplo**

5  
14  
23

---

*Planteador de problemas: Ahmed Shamsul Arefin*

**¡Me encantan los números grandes!**

**El problema**

Una joven japonesa fue a una feria de ciencias en Tokio. Allí conoció a un robot llamado Mico-12, que tenía IA (seguro que sabes lo que es la IA: inteligencia artificial). La joven pensó que podría divertirse con ese robot y le preguntó: "¿Sabes algo de matemáticas?". "¡Sí! Me encantan las matemáticas", respondió el robot.

¡De acuerdo! Te doy un número y tienes que calcular su factorial. Luego, suma los dígitos del resultado. Supongamos que el número es 5. Primero calculas  $5! = 120$ , y después sumas los dígitos:  $1 + 2 + 0 = 3$ . ¿Puedes hacerlo?

"Sí. ¡Puedo hacerlo!" respondió el Robot. "Supongamos que el número es 100, ¿cuál será el resultado?". En este punto, el Robot comenzó a pensar y calcular. Después de unos minutos, la cabeza del Robot se quemó y gritó en voz alta: "Se excedió el límite de tiempo". La chica se rió del Robot y dijo: "La suma es definitivamente 648". "¿Cómo puedes saber eso?", preguntó el Robot a la chica. "Porque soy finalista mundial de ACM y puedo resolver problemas de números grandes fácilmente". Diciendo esto, la chica cerró su computadora portátil y se fue. Ahora, tu tarea es ayudar al Robot con un problema similar.

**La entrada**

El archivo de entrada contendrá uno o más casos de prueba. Cada caso de prueba consta de una línea que contiene un número entero  $n$  ( $n \leq 1000$ ).

### El resultado

Para cada caso de prueba, imprime una línea que contenga el número requerido. Este número siempre será un entero, es decir, menor que  $2^{31}-1$ .

#### Entrada de ejemplo

5  
60  
100

#### Salida de ejemplo

3  
288  
648

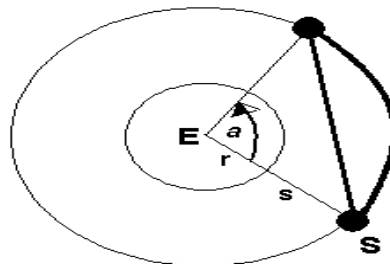
---

*Planteador de problemas: Ahmed Shamsul Arefin*

## satélites

### El problema

El radio de la Tierra es de 6440 kilómetros. Hay muchos satélites y asteroides moviéndose alrededor de la Tierra. Si dos satélites forman un ángulo con el centro de la Tierra, ¿puedes averiguar el radio de la Tierra? *distancia* ¿Entre ellos? Por *distancia* nos referimos a ambos *arco* y *acorde* distancias. Ambos satélites están en la misma órbita. (Sin embargo, tenga en cuenta que giran en una trayectoria circular en lugar de una elíptica).



**E = Earth S = Satellite**

### La entrada

El archivo de entrada contendrá uno o más casos de prueba.

Cada caso de prueba consta de una línea que contiene dos números enteros. **syay** una cuerda "min" o "grados". Aquí **ses** la distancia del satélite a la superficie de la tierra y **aes** el ángulo que forman los satélites con el centro de la tierra. Puede estar en minutos ( ' ) o en grados ( ° ). Recuerda que la misma línea nunca contendrá minutos y grados al mismo tiempo.

### El resultado

Para cada caso de prueba, imprima una línea que contenga las distancias requeridas, es decir, ambas **distancia de arco** y **distancia de la cuerda** respectivamente entre dos satélites en kilómetros. La distancia será un valor de punto flotante con seis dígitos después del punto decimal.

#### Entrada de ejemplo

```
500 30 grados
700 60 min
200 45 grados
```

#### Salida de ejemplo

```
3633.775503 3592.408346
124.616509 124.614927
5215.043805 5082.035982
```

---

*Planteador de problemas: Ahmed Shamsul Arefin*

## Descifra al loco

### El problema

Una vez en BUET, un viejo profesor se volvió completamente loco. Empezó a hablar con palabras extrañas. Nadie podía entender su discurso ni sus conferencias. Finalmente, las autoridades de BUET se metieron en un gran problema. No había manera de mantener a ese hombre trabajando en la universidad. De repente, un estudiante (sin duda era un autor registrado en el capítulo ACM de UVA y tenía un buen puesto en 24-Hour-Online Judge) creó un programa que era capaz de decodificar eso.

Discurso del profesor. Tras su invento, todos volvieron a sentirse cómodos y aquel viejo maestro retomó sus labores cotidianas como antes.

Así que, si alguna vez visitas BUET y ves a un profesor hablando por un micrófono conectado a una computadora IBM equipada con un software de reconocimiento de voz, y los estudiantes toman la clase desde la pantalla de la computadora, ¡no te asustes! Porque ahora tu trabajo es escribir el mismo programa que pueda decodificar el discurso de ese profesor chiflado.

### La entrada

El archivo de entrada contendrá un único caso de prueba, es decir, el mensaje codificado.

El caso de prueba consta de una o más palabras.

### El resultado

Para el caso de prueba dado, imprime una línea que contenga las palabras decodificadas. Sin embargo, no es una tarea tan difícil reemplazar cada letra o símbolo de puntuación por los dos que se encuentran inmediatamente a su izquierda en tu teclado estándar.

#### Entrada de ejemplo

k[r dyt I[o

#### Salida de ejemplo

Cómo estás

---

*Quien plantea problemas: Ahmed Shamsul Arefin*

## ¿Cuántos nodos?

### El problema

Uno de los temas más populares en Estructuras de Datos es el Árbol Binario con Raíz. Si se te dan algunos nodos, sin duda podrás construir el mayor número posible de árboles. Pero si te dan el número máximo de árboles construidos con pocos nodos, ¿puedes determinar cuántos nodos construyeron esos árboles?

### La entrada

El archivo de entrada contendrá uno o más casos de prueba. Cada caso de prueba consta de un número entero  $n$  ( $n \leq 4.294.967.295$ ). Aquí,  $n$  es el número máximo de árboles.

### El resultado

Para cada caso de prueba, imprima una línea que contenga el número real de nodos.

#### Entrada de ejemplo

5  
14  
42

#### Salida de ejemplo

3  
4  
5

---

*Quien plantea problemas: Ahmed Shamsul Arefin*

## El poder de la criptografía

### Fondo

El trabajo actual en criptografía abarca (entre otras cosas) números primos grandes y el cálculo de potencias de números módulo funciones de estos primos. El trabajo en esta área ha dado lugar a la aplicación práctica de resultados de la teoría de números y otras ramas de las matemáticas que antes se consideraban de interés exclusivamente teórico.

Este problema implica el cálculo eficiente de raíces enteras de números.

### El problema

Dado un número entero  $n \geq 1$  y un número entero  $p \geq 1$ , debes escribir un programa que determine  $\sqrt[p]{n}$ , lo positivo  $n^{\text{th}}$  raíz de  $p$ . En este problema, dados dichos números enteros  $n$  y  $p$ , siempre será para un ser de la forma  $k^n$  número entero  $k$  (Este número entero es lo que tu programa debe encontrar).

### La entrada

La entrada consiste en una secuencia de pares de números enteros  $n$  y  $p$  con cada número entero en una línea por sí

mismo. Para todos esos pares  $1 \leq n \leq 200$   $1 \leq p < 10^{101}$ , y existe un número entero  $k$ ,  $1 \leq k \leq 10^9$  de tal manera que  $k^n = p$ .

### El resultado

Para cada par de enteros  $n$  y  $p$  el valor  $\sqrt[n]{p}$  debe imprimirse, es decir, el número  $k$  de tal manera que  $k^n = p$ .

#### Entrada de ejemplo

```
2
16
3
27
7
4357186184021382204544
```

#### Salida de ejemplo

```
4
3
1234
```

## Ruleta romana

El historiador Flavio Josefo relata cómo, durante el conflicto romano-judío del año 67 d. C., los romanos tomaron la ciudad de Jotapata, que él comandaba. Al escapar, Josefo se encontró atrapado en una cueva con 40 compañeros. Los romanos descubrieron su paradero y lo invitaron a rendirse, pero sus compañeros se negaron. Entonces, él les propuso que se mataran entre sí, uno por uno, decidiendo el orden por sorteo. La tradición cuenta que el método para realizar el sorteo consistía en formar un círculo y, comenzando desde un punto determinado, contar hasta que cada tercera persona fuera asesinada. El único superviviente fue Josefo, quien se rindió a los romanos. Esto plantea la pregunta: ¿había practicado Josefo previamente en secreto con 41 piedras en un rincón oscuro, o había calculado matemáticamente que debía adoptar la posición número 31 para sobrevivir?

Tras leer el relato de este espantoso suceso, te obsesiona el miedo a encontrarte en una situación similar en el futuro. Para prepararte ante tal eventualidad, decides escribir un programa para tu ordenador portátil que determine la posición inicial del conteo para asegurarte de ser el único superviviente.

En particular, su programa debe ser capaz de manejar la siguiente variación de los procesos descritos por Josefo.  $norte > 0$  personas se disponen inicialmente en círculo, mirando hacia adentro, y se numeran del 1 al  $norte$ . La numeración del 1 al  $norte$  procede consecutivamente en sentido horario. Su número asignado es 1. Comenzando con la persona número  $i$ , el conteo comienza en sentido horario, hasta que llegamos a la persona número  $k$  ( $k > 0$ ), quien es asesinado inmediatamente. Luego procedemos a contar un número más  $k$  personas en sentido horario, comenzando con la persona inmediatamente a la izquierda de la víctima. El número de persona  $k$  El seleccionado tiene la tarea de enterrar a la víctima y luego regresar a la posición en el círculo que la víctima había ocupado previamente. El conteo procede entonces desde la persona a su izquierda inmediata, con el  $k$  la persona que va siendo asesinada, y así sucesivamente, hasta que solo quede una persona.

Por ejemplo, cuando  $norte = 5$  y  $k = 2$  y  $i = 1$ , el orden de ejecución es 2, 5, 3 y 1. El sobreviviente es 4.

### Entrada y salida

Su programa debe leer líneas de entrada que contengan valores para  $norte$  y  $k$  (en ese orden), y para cada línea de entrada, imprime el número de la persona con la que debe comenzar el conteo para asegurar que seas el único sobreviviente. Por ejemplo, en el caso anterior, la posición inicial segura es 3. La entrada se terminará con una línea que contiene valores de 0 para  $norte$  y  $k$ .

Su programa puede contemplar un máximo de 100 personas participando en este evento.

#### Entrada de ejemplo

```
1 1
1 5
0 0
```

#### Salida de ejemplo

```
1
1
```



## El primo de José

### El problema

El problema de José es de sobra conocido. Para quienes no lo conozcan, entre  $n$  personas numeradas del 1 al  $n$ , cada mes, una persona que se encuentre en círculo será ejecutada, y solo la última persona que quede con vida se salvará. José, con gran astucia, eligió la posición de la última persona con vida, salvando así su vida para poder dar la noticia del suceso.

Aunque muchos buenos programadores se han salvado desde que Joseph difundió esta información, el primo de Joseph introdujo una nueva variante del juego maligno. Este personaje demente es conocido por sus ideas bárbaras y su deseo de librar al mundo de programadores ineptos. Tuvimos que infiltrarnos entre los agentes de la ACM para comprender el funcionamiento de este nuevo juego mortal. Para salvarte de esta práctica perversa, debes desarrollar una herramienta capaz de predecir quién se salvará.

### El proceso destructivo

Las personas son eliminadas en un orden muy particular;  $m$  es una variable dinámica que toma un valor diferente en cada caso, correspondiente a la sucesión de números primos (2, 3, 5, 7...). Así, para eliminar a la  $i$ -ésima persona, el primo de José cuenta hasta el  $i$ -ésimo número primo.

### La entrada

Consta de líneas separadas que contienen  $n$  [1..3501] y termina con un 0.

### El resultado

El resultado consistirá en líneas separadas que contendrán la posición de la persona a la que se le salvará la vida.

#### Entrada de ejemplo

```
6
0
```

#### Salida de ejemplo

```
4
```

**Consulta de números enteros**

Uno de los primeros usuarios de la nueva supercomputadora de BIT fue Chip Diller. Amplió su exploración de potencias de 3 para abarcar desde 0 hasta 333 y exploró diversas sumas de esos números.

«Esta supercomputadora es fantástica», comentó Chip. «Ojalá Timothy estuviera aquí para ver los resultados». (Chip se mudó a un nuevo apartamento en cuanto quedó uno disponible en el tercer piso de los apartamentos Lemon Sky en la Tercera Calle).

**Aporte**

La entrada constará de un máximo de 100 líneas de texto, cada una de las cuales contendrá un único `VeryLongInteger`. Cada `VeryLongInteger` tendrá 100 caracteres o menos y solo contendrá dígitos (ningún `VeryLongInteger` será negativo).

La última línea de entrada contendrá un único cero en una línea aparte.

**Producción**

Tu programa debe mostrar como resultado la suma de los números `VeryLongInteger` proporcionados en la entrada.

**Entrada de ejemplo**

```
123456789012345678901234567890
123456789012345678901234567890
123456789012345678901234567890 0
```

**Salida de ejemplo**

```
370370367037037036703703703670
```

## El decodificador

Escriba un programa completo que decodifique correctamente un conjunto de caracteres en un mensaje válido. Su programa debe leer un archivo dado con un conjunto simple de caracteres codificados e imprimir el mensaje exacto que contienen los caracteres. La clave del código para esta codificación simple es una sustitución de caracteres uno por uno basada en una *manipulación aritmética simple* de la parte imprimible del conjunto de caracteres ASCII.

## Entrada y salida

Por ejemplo: con el archivo de entrada que contiene:

```
1JKJ'pz'{ol'fhyklthyr'vm'{ol'Jvu{yvs'Kh{h'Jvywvyh{pvu5
1PIT'pz'h'fhyklthyr'vm'{ol'Pu{lyuh{pvuhs'I|zplzz'Thjopul'Jvywvyh{pvu5
1KLJ'pz'{ol'fhyklthyr'vm'{ol'Kpnp{hs'Lx|pwtlu{'Jvywvyh{pvu5
```

Tu programa debería imprimir el mensaje:

- \* CDC es una marca registrada de Control Data Corporation.
- \* IBM es una marca comercial de International Business Machine Corporation.
- \* DEC es una marca registrada de Digital Equipment Corporation.

Tu programa debe aceptar todos los conjuntos de caracteres que utilicen el mismo esquema de codificación y debe imprimir el mensaje real de cada conjunto de caracteres.

## Entrada de ejemplo

```
1JKJ'pz'{ol'fhyklthyr'vm'{ol'Jvu{yvs'Kh{h'Jvywvyh{pvu5
1PIT'pz'h'fhyklthyr'vm'{ol'Pu{lyuh{pvuhs'I|zplzz'Thjopul'Jvywvyh{pvu5
1KLJ'pz'{ol'fhyklthyr'vm'{ol'Kpnp{hs'Lx|pwtlu{'Jvywvyh{pvu5
```

## Salida de ejemplo

- \* CDC es una marca registrada de Control Data Corporation.
- \* IBM es una marca comercial de International Business Machine Corporation.
- \* DEC es una marca registrada de Digital Equipment Corporation.

## APÉNDICE B

CÓDIGOS/RUTINAS COMUNES PARA LA PROGRAMACIÓN

---



Este parte de este libro contiene alguno COMÚN  
Códigos, rutinas y pseudocódigos para programadores que se pueden usar FÁCILMENTE  
durante las horas del concurso.<sup>[10]</sup>

Cómo encontrar el máximo común divisor, el mínimo común múltiplo y el factor primo.

```
# incluir<iostream>
# incluir<vector>
# incluir<fstream>
usando el espacio de nombres std;
```

```
entero mcd(int a, int b) {
```

```
    int r=1;
    si(a>b)
    {
        mientras (r!=0)
        {
            r=a%b;
            a=b;
            b=r;
        }
    }
    devolver a;
}
```

```
si no ( b>a)
{
    mientras (r!=0)
    {
        r=b%a;
        b=a;
        a=r;
    }
    devolver b;
}
si no ( a==b)
    devolver a;
}
```

```
int lcm(int a, int b) {
```

```
    devolver a*b/mcd(a,b);
}
```

```
bool esprimo(int a)
```

```
{
    Si (a == 2) devuelve verdadero;
    demás
    {
        for(int i=2;i<a/2+1;++i) if(a%i==0)
            return false; if(i==a/2) return
            true; } } }
```

```
vector<int> primefactors(int a) {
```

```
    vector<int> factores primos;
    para(int i=1;i<=a;++i)
    si(a%i==0&&esprimo(i))
    {
        primefactors.push_back(i); a=a/
        i;i=1;
    }
    devolver factores primos;
}
```

```
int main()
```

```
{
    entero a=5,b=2500;
    out<<"MCD = "<<mcd(a,b)<<endl;
    out<<"MCM = "<<mcm(a,b)<<endl; if(!
    isprime(b)) out<<"no es primo"<<endl;
```

```
    vector <int> p =
    primefactors(b);
    para(int i=0;i<p.size();++i)
    salida<<p[i]<<endl;
    devolver 0;
}
```

Hallar recursivamente el MCD mediante el algoritmo de Euclides.

```
//*****          MCD utilizando un método recursivo          ***** /
# incluir<stdio.h>
entero sin signo Euclid_g_c_d(entero sin signo a, entero sin signo b){ si(b==0)

    devolver a;
    demás
        devolver Euclid_g_c_d(b,a%b);
}
int main(){
    entero sin signo a, b, mcd;
    mientras( scanf("%d%d",&a,&b)==2){
        mcd=Euclid_g_c_d(a,b);
        printf("%d",gcd);
    }
    devolver 0;
}
```

Otra aplicación del MCD es el algoritmo extendido de Euclides. Podemos resolver la fórmula  $d = ax + by \Rightarrow \text{mcd}(a,b) = ax + by$ . Este algoritmo toma como entradas a y b... y produce como salida d(mcd de a,b) y el valor de la raíz x e y.

```
//*****          Algoritmo extendido de Euclides          ***** //
# incluir<stdio.h>
entero largo sin signo d,x,y;
void Extended_Euclid(unsigned long a, unsigned long b){
    largo sin signo x1;
    si(b>a){
        x1=a;    //si b>a, entonces usé esta condición if // el
        a=b;    resultado es correcto pero x e y están intercambiados
        b=x1;
    }
    si(b==0){
        d=a;
        x=1;
        y=0;
        devolver;
    }
    Euclid extendido(b,a%b); d = d;

    x1 = x-(a/b) * y; x = y;

    y = x1;
}
int main(){
    entero largo sin signo a,b;          //      d = mcd(a,b) = ax+by
    mientras(scanf("%lu %lu", &a, &b)==2)
    { Extended_Euclid(a,b);
      printf("%lu %lu %lu\n",d,x,y);}return 0;}
```

## MCD de N números:

Este programa requiere un número que indica la cantidad de números cuyo máximo común divisor (MCD) se desea calcular y mostrar. El programa acepta datos hasta el final del archivo.

```
//***** MCD para n números ***** //
# incluir<stdio.h>
int main(){
    mcd largo,a,b,n,i;
    mientras(scanf("%ld",&n)==1){
        scanf("%ld%ld",&a,&b);
        i=2;
        mientras(i<n){
            mcd=g_c_d(a,b);
            a=máximo común divisor;

        i++;
            scanf("%ld",&b);
        }
        mcd=g_c_d(a,b);
        printf("%ld\n",gcd);
    }
    devolver 0;
}
```

## MCM (Mínimo Común Múltiplo)

También es un tema importante en programación. Supongamos que diferentes relojes están configurados en 12 puntos, y cada reloj tiene su propio tiempo para completar el ciclo. ¿Cuándo volverán todos los relojes a estar en 12 puntos? Este tipo de problema se puede resolver con el mínimo común múltiplo (MCM). El MCM se implementó con la ayuda del máximo común divisor (MCD).

**MCM (m, n)** =  $[m / \text{MCD}(m, n)] * n$ ; O bien,  
 $D1 = m / \text{MCD}(m, n);$   
 $D2 = n / \text{MCD}(m, n);$   
 $\text{MCM}(m, n) = D1 * D2 * \text{MCD}(m, n);$

Aquí está el código de algunos números enteros MCM.

```
MCD largo(a largo, b largo){
    si(b==0)
        devolver a;
    demás
        devolver MCD(b,a%b);}
MCM largo(a largo,b largo){
```

```

        devolver (a/MCD(a,b)*b);
    }
    int main(){
        a,b largos;
        int n,i;
        mientras(scanf("%d",&n)==1){
            si(n==0) salir;
            scanf("%ld",&a);
            para(i=1;i<n;i++){
                scanf("%ld",&b);
                a=MCM(a,b);
            }
            printf("%ld\n",a);
        }
        devolver 0;}

```

## Factoriales

$n! = n * (n-1) * (n-2) * (n-3) \dots \text{hasta } 1$ .  $5! = 5 * (5-1) * (5-2) * (5-3) * 2 * 1 = 120$ . Este es un proceso recursivo.

```

/** Factorial por recursión ***** //
# incluir<stdio.h>
fac largo(x largo){
    si(x==0)
        devolver 1;
    devolver x * fac(x-1);
}
/*****/ int main(){

    largo n,res;
    mientras(scanf("%ld",&n)==1){
        res=fac(n);
        printf("%ld\n",res);
    }
    devolver 0;}

```

Esto es muy fácil, pero hay algo que debemos saber: los factoriales aumentan y se multiplican. El 12º factorial es 479001600, que es muy grande, así que ¿puedes imaginar el factorial de 1000!...? El programa anterior puede calcular hasta 12º factoriales, por lo tanto, para menos de 12º podemos usarlo. Pero por cuestiones de tiempo, en ACM siempre intentamos evitar el proceso recursivo.



## Factorial iterativo

```
//***** Factorial por multiplicación normal *****//
# incluir<stdio.h>
fac largo(x largo){
    largo i,res=1;
    para(i=1;i<=x;i++){
        res*=i;
    }
    devolver res;
}
int main(){
    largo n,res;
    mientras(scanf("%ld",&n)==1){
        res=fac(n);
        printf("%ld\n",res);
    }
    devolver 0;
}
```

Entonces, ¿cómo podemos calcular el factorial de números grandes? Aquí hay un código que puede mostrar los factoriales hasta 1000. ¡Y este es un problema de ACM (623-500)!). ¡Este código también es muy eficiente! Precalculamos todos los 1000 factoriales en una matriz bidimensional. En la función principal, simplemente se toma la entrada y luego se imprime.

## ¡Factorial de un número grande!

```
# incluir<stdio.h>
# incluir<string.h>
carácter f[10000];
char factorial[1010][10000]; void
multiply(int k){
    entero cin, suma, i;
    int len = strlen(f); cin=0;

    i=0;
    mientras(i<len){
        suma = cin + (f[i] - '0') * k; f[i] =
            (suma % 10) + '0'; i++;

        cin = suma/10;
    }
    mientras(cin>0){
        f[i++] = (cin%10) + '0'; cin/=10;
    }
    f[i]='\0';
    para(int j=0;j<i;j++){
        factorial[k][j]=f[j];
    }
    factorial[k][i]='\0';}
```

```

void fac(){
    entero k;
    strcpy(f,"1");
    para(k=2;k<=1000;k++)
        multiplicar(k);
}
void print(int n){
    entero i;
    int len = strlen(factorial[n]); printf("%d!\n",n);
    para(i=len-1;i>=0;i--){
        printf("%c",factorial[n][i]);
    }
    printf("\n");
}
int main(){
    entero n;
    factorial[0][0]='1';
    factorial[1][0]='1';
    fac();
    mientras(scanf("%d",&n)==1){
        imprimir(n);
    }
    devolver 0;
}

```

## Frecuencias factoriales

Ahora bien, ¿cuántos dígitos tiene el factorial de N números...? Sí, podemos contarlos mediante el proceso anterior. ¡Pero podemos hacerlo de forma sencilla!

```

//*****      ¿Cuántos dígitos tiene N factoriales?      *****//
# incluir<stdio.h>
# incluir<math.h>
/*****/ double
count_digit(long a){
    suma doble;
    largo yo;
    Si (a == 0) devuelve 1;
    demás{
        suma=0;
        para(i=1;i<=a;i++){
            suma+=log10(i);
        }
        devolver piso(suma)+1;
    }
}
/*****/ int main(){

    suma doble;
    n larga;

```

```

    mientras(scanf("%ld",&n)==1){
        suma=contar_dígitos(n);
        printf("%.0lf\n",sum);
    }
    devolver 0;
}

```

### ¿Cuántos ceros finales hay en los factoriales?

Sabemos que los factoriales de cualquier número tienen cierta cantidad de ceros (0) al final... por ejemplo,  $17! = 355687428096000$ . Tiene 3 dígitos cero al final. Así que ahora veremos cuántos ceros finales tiene N factorial.

```

//*****                                ¿Cuántos ceros finales hay en N!?
* * * * * /*****//
# incluir<stdio.h>
cero largo(número largo, factor largo){
    total largo,deno;
    Si (número == 5) devuelve 1;
    total = 0;
    deno=factor;
    mientras(deno<número){
        total+=número/denominación;
        deno*=factor;
    }
    devolver total;
}
/*****/ int main(){

    N largo,c2,c1;
    mientras(scanf("%ld",&N)==1){
        c1=cero(N,2);
        c2=cero(N,5);
        if(c1<c2) printf("%ld\n",c1); else
        printf("%ld\n",c2);
    }
    devolver 0;
}

```

### Gran Mod

Sin embargo, no siempre nos interesan las respuestas completas. A veces, el resto es suficiente para nuestros propósitos. En ese caso, utilizamos la aritmética modular. La clave para una aritmética modular eficiente reside en comprender cómo funcionan las operaciones básicas de suma, resta y multiplicación sobre un módulo dado.

Suma:  $(x + y) \bmod n \dots = ((x \bmod n) + (y \bmod n)) \bmod n$ . Resta:  $(xy) \bmod n \dots = ((x \bmod n) - (y \bmod n)) \bmod n$ .

Podemos convertir un número negativo módulo  $n$  en un número positivo sumándole un múltiplo de  $n$ .

Multiplicación:  $xy \bmod n \dots = (x \bmod n)(y \bmod n) \bmod n$ .

**¿Qué es  $x^y \bmod n$ ?** La exponenciación es simplemente una multiplicación repetida,  $(x \bmod n)$ , y módulo  $n$ . Entonces, si  $x$  e  $y$  son muy grandes, iguales a 32 bits, también podemos calcularlo sin desbordamiento.

```
/* Modificación grande */
# incluir<stdio.h>
cuadrado largo (long s){
    devolver s*s;
}
largo bigmod(largo b, largo p, largo m){
    si (p == 0)
        devolver 1;
    si no, si (p%2 == 0)
        return cuadrado( bigmod (b,p/2,m)) % m; // cuadrado(x) = x * x else

        devolver ((b % m) * bigmod( b,p-1,m)) % m;
}
int main(){
    b,p,m,suma largo;
    mientras(scanf("%ld%ld%ld",&b,&p,&m)==3){
        suma = bigmod( b, p, m);
        printf("%ld\n", suma);
    }
    devolver 0;}
```

## Conversiones numéricas - Entero      Binario

### Convertir un número entero en un número binario.

```
int inttobin(unsigned int a,int
* papelera)
{
    entero uno=0;
    entero sin signo c=1;
    int i;
    para(i=31;i>=0;i--)
    {
        uno = uno + (a&c); bin[i] =
        (a&c)?1:0;
        c<<=1;
    }
    devolver uno; }
```

### Convertir un número binario en un número binario

```
entero sin signo creatnum(int *num) {
    entero i;
    entero sin signo a=0;
    para(i=0;i<32;i++)
    {
        a = a | num[i];
        si(i!=31)
        a <<= 1;
    }
    devolver a;
}
```

Entero      maleficio      Entero

```
carácter hexadecimal[100];
entero num = 4095;
sprintf(hex,"%X",num); // convierte el número hexadecimal en decimal en mayúsculas
sprintf(hex,"%x",num); // convierte el número hexadecimal en decimal en minúsculas
sscanf(hex,"%x",&num); // convierte el número hexadecimal en entero num
```

Número entero a cualquier base.

El siguiente código te ayudará a convertir cualquier número entero en cualquier base desde 2 hasta 36.

```
# incluir<string.h>
# incluir<stdio.h>
# define MAXDGT 200          /* número máximo de dígitos */

/* ***** */
/*      Una función de intercambio general que intercambia dos          */
/*      objetos. Entrada: Dos objetos.                                  */
/*      Retorno: Ninguno                                                */
/* ***** */
/* ***** plantilla ***** */
<clase T>

void swap(T &x, T &y) { T
tmp=x; x=y;          y=tmp;    };

/* ***** */
/*      Una función general para invertir cadenas que invierte la cadena pasada y */
/*      almacena la cadena resultante en el parámetro. Entrada: Una cadena      */
/* ***** */
/*      Retorno: Ninguno */
/* ***** */ void
revstr(char *str)
{
    int i=0,l=strlen(str);
    mientras(i<l)
        intercambiar(str[i++],str[--l]);
}

/* ***** */
/*      Función general de conversión de bases.                               */
/*      Entrada: Un número n y una base b.                                   */
/*      Devuelve: Una matriz de caracteres que contiene el número n en base b */
/* ***** */ char
*itob(long int n,int b=10)
{
    carácter num[MAXDGT];
    entero j, signo;
    registrar int i=0;
```

```

    si( (sign=n) <0 )
        n = -n;
hacer  {      j=n%b;
        num[i++]=j<10 ? (j+'0'): ('A'+j-10); }while((n/=b)!=0);

    si(signo < 0)          num[i++]='-';

    num[i]='\0';
    revstr(num);
    devolver num;
}

/* Ejemplo principal */
main(void)
{ printf(itob(71],36));      }

```

### De decimal a romano

El sistema de numeración romano es muy útil. A menudo necesitamos convertir un número decimal a número romano. En el sistema romano se utilizan los siguientes símbolos.

i, v, x, l, c, m para 1, 5, 10, 50, 100 y 500 respectivamente.

# incluir<stdio.h>

```

/* ***** */          * /
    Convertir el número 1 al 10          * /
/*      Entrada: Un número entero          * /
/*      Retorno: Ninguno          * /
/* ***** */          * /

void unit(int n)
{
    interruptor(n){
        caso 3:    printf("i");
        caso 2:    printf("i");
        caso 1:    printf("i"); break;
        caso 4:    printf("i");
        caso 5:    printf("v"); break;
        caso 6:    printf("vi"); break;
        caso 7:    printf("vii"); break;
        caso 8:    printf("viii"); break;
        caso 9:    printf("ix"); break;
    }
}

/* ***** */          * /
    Convertir el número 10 a 100          * /

```

```

/*      Entrada: Un número entero      */
/*      *****      */
void diez(int n)
{
    interruptor(n){
        caso 3:    printf("x");
        caso 2:    printf("x");
        caso 1:    printf("x"); break;
        caso 4:    printf("x");
        caso 5:    printf("l"); break;
        caso 6:    printf("lx"); break;
        caso 7:    printf("lxx"); break;
        caso 8:    printf("lxxx"); break;
        caso 9:    printf("xc"); break;
    }
}

/*      *****      */
/*      Convertir el número 100 a 500.      */
/*      Entrada: Un número entero.      */
/*      *****      */
void hnd(int n)
{
    interruptor(n){
        caso 3:    printf("c");
        caso 2:    printf("c");
        caso 1:    printf("c"); break;
        caso 4:    printf("c");
        caso 5:    printf("M"); break;
    }
}

/*      *****      */
/*      Convierte un número entero al sistema romano. Entrada:      */
/*      Un número entero.      */
/*      *****      */
void
roman(int n)
{
    int a,i;
    si(n>=500)
    {
        a=n/500;
        para(i=1;i<=a;i++)
            printf("M");
    }
    n=n%500;
    hnd(n/100);
    n=n%100;
    diez(n/10);
    unidad(n%10);
} principal(void)
{
    romano(390);
    devolver 0;
}

```

### El problema de Josefo

Flavio Josefo fue un famoso historiador del siglo I. Cuenta la leyenda que no habría alcanzado la fama sin su talento matemático. Durante la guerra judeo-romana, se encontraba entre un grupo de 41 rebeldes judíos atrapados en una cueva por los romanos. Prefiriendo el suicidio a la captura, los rebeldes decidieron formar un círculo y, avanzando a su alrededor, matar a cada tercera persona que quedara hasta que no quedara nadie. Pero Josefo, junto con un cómplice no acusado, no quería participar en esa locura suicida; así que calculó rápidamente la posición de él y su amigo en el círculo vicioso.

```
#incluir <stdio.h>
# define MAX 150

/** Implementación de una cola circular **/

entero cola[MÁX];
int e,f,r;

/** _f es frontal; _r es posterior y _e es el número de elemento **/

/* ***** */ /* Un
algoritmo general de empuje que actúa sobre una cola */ /*
***** */

void push(int i)
{
    r++;
    e++;
    si(r>=MAX)
        r=1;
    cola[r] = i;
}

/* ***** */ /* Un
algoritmo pop general actúa sobre una cola */ /*
***** */

entero pop(void)
{
    mi--;
    int i=cola[f];
    f++;
    si(f>=MAX)
        f = 1;
    devolver i;
}
```



```
/** Fin de la cola circular **/  
/* ***** */  
/* Una función general de joshuf. /* Esta */  
función comienza eliminando /* el primer */  
elemento /* y devuelve el superviviente */  
/* ***** */  
  
int joshups(int n,int v) {  
  
    registrar int i;  
    e=n;  
    para(i=1;i<=n;i++)  
        cola[i] = i;  
    f = 1; // 0 para servir el primer elemento sobreviviente. r =  
    n; // n+1 para servir el primer elemento sobreviviente. i = 0;  
  
    si(n > 1)  
        estallido();  
    mientras(e!=1)  
    {  
        si(i!=v)  
        {  
            i++;  
            empujar(pop());  
        }  
        demás  
        {  
            estallido();  
            i = 0;  
        }  
    }  
    devolver cola[f];  
}  
  
/*función principal de ejemplo*/  
  
principal(void)  
{  
    entero i,m;  
    scanf("%d",&i);  
    mientras(i)  
    { m=1;  
        while((joshups(i,m++)) != 13 );  
        printf("%d",m);  
        putchar('\n');  
        scanf("%d",&i);  
    }  
  
    devolver 0; }
```

## Combinaciones

```

/* ***** */
/*          Calcular nCm                      */
/*          Entrada: Dos números enteros nm.  */
/*          Salida: El valor de nCm.          */
/* ***** */

double nCr(int n,int m) {

    entero k;
    registrar int i,j;
    doble c,d;

    c=d=1;
    k=(m>(nm))?m:(nm);
    para(j=1,i=k+1;(i<=n);i++,j++) {

        c*=i;
        d*=j;
        si( !fmod(c,d)      && (d!=1) )
        {   c/=d;
            d=1;
        }
    }

    devolver c;
}

/* Una función principal de ejemplo */
main(void)
{
    entero n,m;

    while(scanf("%d%d",&n,&m)!=EOF)
        printf("%.0lf\n",nCr(n,m)); return 0;
}

```

**Otra forma de calcular el  $nCm$  es utilizando el triángulo de Pascal.**

```

# incluir<stdio.h>
# define MAXTRI 50

paso int largo sin firmar[MAXTRI][MAXTRI];

void pascals(int n)
{

```